# Seguridad en Base de Datos - Control de Acceso

Los ejercicios 1 a 3 se resuelven en papel, según el estándar SQL. Los ejercicios 4 a 9 se resuelven en PostgreSQL: escriba las sentencias, prediga el resultado y **verifíquelo contra el servidor**. Para cambiar de usuario dentro de la misma sesión use `SET ROLE usuario;` y para volver, `RESET ROLE;` (en DataGrip, DBeaver o pgAdmin cada consola es una sesión distinta). En psql los privilegios se ven con `\du` (roles), `\dn+` (esquemas), `\dp tabla` (tablas) y `\d+ tabla` (incluye políticas); desde un cliente gráfico use las consultas equivalentes que siguen.

**Consultas equivalentes a los comandos de psql (catálogo de PostgreSQL)**

```
-- ¿Quién soy?  SET ROLE cambia current_user; session_user no cambia
SELECT current_user, session_user;

-- Roles, atributos y membresías  (\du)
SELECT r.rolname, r.rolsuper, r.rolcreaterole, r.rolcreatedb, r.rolcanlogin,
       r.rolbypassrls,
       ARRAY(SELECT g.rolname FROM pg_auth_members m
             JOIN pg_roles g ON g.oid = m.roleid
             WHERE m.member = r.oid) AS miembro_de
FROM pg_roles r
WHERE r.rolname NOT LIKE 'pg\_%'
ORDER BY 1;

-- Esquemas, dueño y privilegios  (\dn+)
SELECT nspname AS esquema, pg_get_userbyid(nspowner) AS dueno,
       nspacl AS privilegios
FROM pg_namespace
WHERE nspname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
ORDER BY 1;

-- Privilegios sobre tablas, vistas y secuencias, en el formato de \dp
-- (rol=privilegios/otorgante; NULL = sólo el dueño tiene privilegios)
SELECT n.nspname AS esquema, c.relname AS objeto, c.relkind AS tipo,
       pg_get_userbyid(c.relowner) AS dueno, c.relacl AS privilegios
FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r', 'v', 'S') AND n.nspname = 'ventas'
ORDER BY 1, 2;

-- Lo mismo, una fila por privilegio (más legible, estándar SQL)
SELECT table_name, grantor, grantee, privilege_type, is_grantable
FROM information_schema.table_privileges
WHERE table_schema = 'ventas'
ORDER BY 1, 3, 4;

-- Privilegios de columna, sólo los otorgados con GRANT priv (columna, ...)
SELECT c.relname AS tabla, a.attname AS columna, a.attacl AS privilegios
FROM pg_attribute a JOIN pg_class c ON c.oid = a.attrelid
     JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'ventas' AND a.attacl IS NOT NULL
ORDER BY 1, 2;

-- ¿Puede este rol hacer esto?
SELECT has_table_privilege('ana', 'ventas.factura', 'SELECT');
SELECT has_schema_privilege('ana', 'ventas', 'USAGE');
```

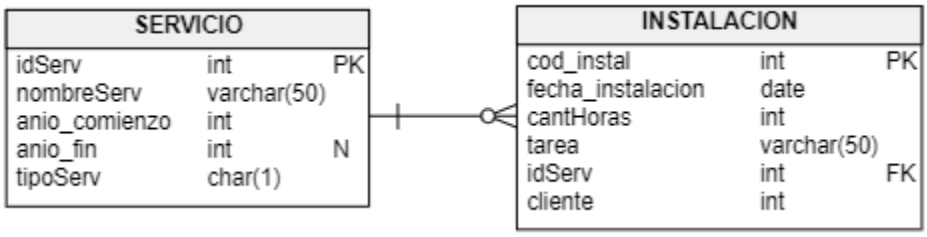
# Seguridad en Base de Datos - Control de Acceso

```
-- RLS: ¿activada? ¿forzada al dueño?  y políticas definidas  (\d+ tabla)
SELECT  relname,  relrowsecurity  AS  rls_activa,  relforcerowsecurity  AS
rls_forzada
FROM pg_class
WHERE relnamespace = 'ventas'::regnamespace AND relkind = 'r';

SELECT tablename, policyname, permissive, roles, cmd, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas';
```

## Ejercicio 1

Considere la siguiente fracción del esquema del usuario db_instal que registra datos de las instalaciones de servicios a clientes




1- Suponga que db_instal ejecuta las siguientes sentencias:

```
GRANT SELECT ON instalacion TO adm WITH GRANT OPTION;
GRANT UPDATE ON instalacion TO adm WITH GRANT OPTION;
GRANT DELETE ON instalacion TO adm;
```

2- Luego el usuario adm ejecuta las siguientes sentencias:

```
GRANT SELECT ON instalacion TO doc;
GRANT UPDATE(cantHoras, tarea) ON instalacion TO doc WITH GRANT OPTION;
GRANT DELETE ON instalacion TO doc;
```

a) Realice el grafo de permisos correspondiente a las sentencias indicadas en 1 y 2. Analice qué sucede en cada caso y si alguna de ellas arroja algún error

b) Establezca los permisos que conserva el usuario doc sobre la tabla instalacion, si el usuario db_instal ejecuta las siguientes sentencias (considere la posibilidad de error):

```
REVOKE SELECT ON instalacion FROM adm CASCADE;
REVOKE UPDATE(tarea) ON instalacion FROM adm CASCADE;
```

## Ejercicio 2

Considere la base de Voluntarios y suponga que las siguientes acciones son ejecutadas por un usuario U0 con privilegios de administrador, salvo que se especifique otro usuario. En cada caso indique la sentencia correspondiente según el estándar SQL para control de acceso.
Tenga en cuenta que las operaciones se realizan en el orden indicado y explique en caso de que algún ítem no pueda ser ejecutado.

a) Conceder al usuario U1 todos los privilegios sobre la tabla Institución, con posibilidad de concederlos a otros usuarios.

# Seguridad en Base de Datos - Control de Acceso

b) Permitir que el usuario U2 sólo consulte los datos de la tabla Voluntario.
c) Permitir que U2 autorice a U3 para que pueda insertar en la tabla Voluntario.
d) Dar a todos los usuarios del sistema privilegios de inserción y actualización sobre la tabla Tarea. ¿Cuál sería la diferencia si se incluye o no la cláusula WITH GRANT OPTION?
e) Retirar el privilegio de borrado sobre Institución a U1.
f) Quitar el privilegio de inserción a todos los usuarios sobre la tabla Tarea ¿Qué usuario podría entonces insertar datos en esa tabla?
g) Crear un rol ins_prov con capacidad de insertar datos de provincias y países.
h) Crear dos usuarios U3 y U4 y asignarles el rol ins_prov.
i) Permitir que los usuarios con el rol ins_prov puedan también actualizar el nombre del voluntario.
j) Eliminar el rol ins_prov ¿qué sucede con los usuarios U3 y U4?

### Ejercicio 3

Suponga que el usuario A ha creado la tabla Usuario(nro_u, nombre, tarea) y luego ejecuta los siguientes comandos SQL:

```
GRANT INSERT ON Usuario TO B WITH GRANT OPTION;
GRANT SELECT ON Usuario TO B WITH GRANT OPTION;
GRANT SELECT ON Usuario TO C;
```

a) Indique quiénes pueden ejecutar exitosamente los siguientes comandos:

1. `SELECT * FROM A.Usuario WHERE nro_u='C';`
2. `INSERT INTO A.Usuario VALUES ('C','Gerente', 'Control');`
3. `GRANT SELECT ON A.Usuario TO D;`

b) Establezca la habilitación (o no) para realizar los siguientes comandos:

1. `B: GRANT INSERT ON Usuario TO D;`
2. `A: REVOKE INSERT ON Usuario FROM B CASCADE;`

c) ¿Cuáles permisos conservan los usuarios después de lo anterior?

### Ejercicio 4

Un administrador quiere que el usuario `lucia` pueda conectarse, crear tablas en el esquema `ventas` y consultar la tabla `ventas.cliente`. Para eso ejecuta, como superusuario:

```
CREATE ROLE lucia;
GRANT CREATE TABLE TO lucia;
GRANT SELECT ON ventas.cliente TO lucia;
```

a) Indique qué sentencias fallan en PostgreSQL y por qué. Reescriba la secuencia para que `lucia` logre las tres cosas. (Pista: atributos del rol vs. privilegios sobre objetos; privilegios sobre el esquema.)

b) Con la secuencia corregida, `lucia` ejecuta `SELECT * FROM ventas.cliente;` y obtiene `ERROR: permission denied for schema ventas`. Explique por qué, si tiene SELECT sobre la tabla, y corríjalo.

c) Más tarde `lucia` recibe INSERT sobre `ventas.cliente`, cuya clave es `id serial`. Ejecuta `INSERT INTO ventas.cliente (nombre) VALUES ('Acme');` y obtiene `permission denied for`

# Seguridad en Base de Datos - Control de Acceso

`sequence cliente_id_seq`. ¿Qué privilegio falta y por qué? ¿Cómo lo otorgaría para todas las secuencias del esquema de una sola vez?

d) ¿Qué diferencia hay entre `CREATE USER lucia` y `CREATE ROLE lucia`? Compruébelo con `\du`.

e) Para cada capacidad indique si en PostgreSQL es un **atributo del rol** o un **privilegio sobre un objeto**, y escriba la sentencia que la otorga a `lucia`: (i) crear bases de datos; (ii) conectarse a la base `facturacion`; (iii) ejecutar la función `calcular_total()`; (iv) definir una clave foránea que apunte a `ventas.cliente`; (v) crear otros roles; (vi) no estar sujeta a políticas de seguridad a nivel de fila.

## Ejercicio 5

Al ejecutar `\dp ventas.factura` en psql se obtiene la siguiente columna de privilegios (formato `rol=privilegios/otorgante`; letras: `a` INSERT, `r` SELECT, `w` UPDATE, `d` DELETE, `D` TRUNCATE, `x` REFERENCES, `t` TRIGGER; un `*` después de una letra indica grant option):

```
adm=arwdDxt/adm
ana=r/adm
bruno=ar*w/adm
carla=r/bruno
=r/adm
```

a) Indique, para cada rol, qué operaciones puede ejecutar sobre `factura` y cuáles puede propagar a otros.

b) Escriba la secuencia de sentencias GRANT que produjo ese estado, indicando quién ejecuta cada una. ¿Quién es el propietario de la tabla? ¿Cómo lo sabe?

c) ¿Qué significa la línea `=r/adm`? Si mañana se crea el usuario `diego` sin ningún GRANT, ¿podrá consultar `factura`?

d) `adm` ejecuta `REVOKE SELECT ON ventas.factura FROM bruno;`. Indique qué ocurre y por qué. Luego muestre cómo quedaría `\dp` si en cambio agrega `CASCADE`.

e) Haga que únicamente `adm`, `ana` y `bruno` puedan consultar la tabla. ¿Puede `adm` resolverlo solo, sin alterar los privilegios de `bruno`? ¿Quién tiene que intervenir?

f) Dibuje el grafo de concesión del estado inicial y vuelva a dibujarlo después del REVOKE ... CASCADE del ítem d).

## Ejercicio 6

La tabla `rrhh.empleado(legajo, nombre, sector, sueldo, cuil)` pertenece al usuario `rrhh_adm`. El usuario `consulta` debe poder ver `legajo`, `nombre` y `sector` de todos los empleados, y nada más.

a) Resuélvalo con una vista: escriba el `CREATE VIEW` y los GRANT necesarios. ¿Necesita `consulta` algún privilegio sobre `empleado` o sobre el esquema `rrhh`? Justifique con el modelo de PostgreSQL (¿con los privilegios de quién se verifica el acceso a la tabla base?).

b) Resuélvalo ahora sin vistas, con privilegios de columna. Compare ambas soluciones ejecutando como `consulta`: `SELECT * FROM rrhh.empleado;`, `SELECT legajo, nombre FROM rrhh.empleado;` y `SELECT sueldo FROM rrhh.empleado;`.

c) El usuario `analista` tiene SELECT sobre `empleado` (otorgado por `rrhh_adm` sin grant option), crea la vista `rrhh.empleado_sector` sobre ella y ejecuta `GRANT SELECT ON rrhh.empleado_sector TO consulta;`. ¿Es válido según el estándar SQL? ¿Y en PostgreSQL? Verifíquelo.

# Seguridad en Base de Datos - Control de Acceso

d) `rrhh_adm` ejecuta `REVOKE SELECT ON rrhh.empleado FROM analista;`. ¿Qué ocurre con la vista `empleado_sector` según el estándar SQL? ¿Y en PostgreSQL? ¿Qué obtiene `consulta` al consultarla? ¿Y `analista`?

e) Cree la vista `rrhh.mis_datos AS SELECT * FROM rrhh.empleado WHERE nombre = current_user;` como `rrhh_adm` y concédala a `consulta`. ¿Qué devuelve cuando la consulta `rrhh_adm`? ¿Y `consulta`? ¿Qué cambia si la vista se crea `WITH (security_invoker = true)`?

f) ¿Qué problema resuelve `WITH (security_barrier = true)` en una vista con WHERE? Describa un ejemplo de consulta que, sin esa opción, podría revelar filas ocultas.

### Ejercicio 7

La base de facturación tiene el esquema `ventas` con las tablas `cliente`, `producto`, `factura` y `linea_factura` (todas con clave `serial`). La usarán: tres vendedores (`ana`, `bruno`, `carla`) que cargan y consultan facturas; un contador (`diego`) que sólo consulta; la aplicación web (`app_web`), que carga y consulta; y el administrador `ventas_adm`, dueño de todos los objetos.

a) Diseñe los roles: cuáles son de grupo (sin LOGIN), cuáles de login, y qué privilegios recibe cada grupo. Dibuje el grafo personas → roles de grupo → objetos.

b) Escriba el script completo: `CREATE ROLE`, GRANT sobre el esquema, las tablas y las secuencias, y las membresías. Use `ON ALL TABLES IN SCHEMA` / `ON ALL SEQUENCES IN SCHEMA`.

c) Un mes después `ventas_adm` crea la tabla `ventas.promocion`. ¿Pueden consultarla `diego` y los vendedores? ¿Por qué? Agregue al script lo necesario (`ALTER DEFAULT PRIVILEGES`) para que las tablas futuras salgan con los permisos correctos.

d) `carla` pasa a ser supervisora: además de cargar, debe poder borrar facturas y líneas. Resuélvalo con una sola sentencia sobre `carla`, sin tocar privilegios de objetos. ¿Qué tuvo que crear antes?

e) `ana` deja la empresa. Escriba las sentencias necesarias. Ejecute `DROP ROLE ana;` directamente y explique el error, si lo hay. (Pista: ¿`ana` es dueña de algo?, ¿tiene privilegios concedidos?; vea `REASSIGN OWNED BY` y `DROP OWNED BY`.)

f) Verifique el resultado con `\du`, `\dn+ ventas` y `\dp ventas.*`. ¿Qué devuelve `SELECT has_table_privilege('diego', 'ventas.factura', 'INSERT');`? ¿Y con `'SELECT'`?

g) Alguien propone que `app_web` se conecte directamente como `ventas_adm` "para simplificar". Dé al menos dos razones concretas, basadas en lo visto en la clase, para rechazar la propuesta.

### Ejercicio 8

Sobre el esquema del ejercicio anterior se requiere que cada vendedor vea y modifique **únicamente sus propios clientes**. La tabla es `ventas.cliente(id serial, nombre text, vendedor name)`, donde `vendedor` es el nombre del rol que lo atiende. Llame `ventas_carga` al rol de grupo de los vendedores.

a) ¿Por qué no alcanza con lo hecho en el ejercicio 7? Indique qué puede hacer `bruno` hoy sobre los clientes de `ana`.

b) Active la seguridad a nivel de fila sobre `cliente` y escriba una política para `ventas_carga` que limite las filas por `vendedor = current_user`, tanto para leer como para escribir.

c) Prediga el resultado de cada sentencia ejecutada por `ana` (suponga que el cliente 1 es de `ana` y el 2 de `bruno`) y luego verifíquelo:

1. `SELECT count(*) FROM ventas.cliente;`

# Seguridad en Base de Datos - Control de Acceso

2. `UPDATE ventas.cliente SET nombre = 'X' WHERE vendedor = 'bruno';`
3. `INSERT INTO ventas.cliente (nombre, vendedor) VALUES ('Nuevo', 'bruno');`
4. `UPDATE ventas.cliente SET vendedor = 'bruno' WHERE id = 1;`
5. `DELETE FROM ventas.cliente WHERE id = 2;`

d) `diego` (sólo consulta) ejecuta `SELECT * FROM ventas.cliente;` y obtiene 0 filas. ¿Por qué, si tiene SELECT sobre la tabla? Escriba la política que le permite ver todos los clientes sin poder modificar ninguno.

e) `ventas_adm` ejecuta la misma consulta y ve todas las filas, aunque no hay ninguna política para él. ¿Por qué? ¿Qué debería hacer para que las políticas también se le apliquen?

f) Se agrega una regla: nadie puede ver los clientes con `bloqueado = true`, ni siquiera su propio vendedor. Escríbala como política `RESTRICTIVE` y explique por qué una política `PERMISSIVE` adicional no serviría.

g) Modifique las políticas para que los vendedores puedan **ver todos** los clientes pero **insertar y modificar sólo los propios**. ¿Alcanza con una sola política? (Pista: una política puede ser `FOR SELECT`, `FOR INSERT` o `FOR UPDATE`, y `USING` y `WITH CHECK` no tienen por qué ser iguales.)

h) `app_web` se conecta con un único rol pero atiende a los tres vendedores. Adapte la política para que use `current_setting('app.vendedor', true)` y muestre qué debe ejecutar la aplicación al inicio de cada transacción. ¿Qué riesgo aparece si olvida hacer `RESET` al devolver la conexión al pool?

i) Muestre qué lista `\d+ ventas.cliente` y consulte `pg_policies`. Explique el contenido de las columnas `permissive`, `roles`, `cmd`, `qual` y `with_check`.

### Ejercicio 9

Indique si cada afirmación es verdadera o falsa para PostgreSQL y justifique brevemente. Cuando pueda, compruébelo en psql.

1. `REVOKE` sin `CASCADE` ni `RESTRICT` siempre tiene éxito.
2. Si un usuario recibió el mismo privilegio de dos otorgantes distintos, un `REVOKE ... CASCADE` ejecutado por uno de ellos lo deja sin el privilegio.
3. Un superusuario está sujeto a las políticas de seguridad a nivel de fila como cualquier otro rol.
4. Una vista con `WHERE vendedor = current_user` es equivalente a una política de RLS sobre la tabla.
5. Conceder SELECT sobre una tabla a `PUBLIC` afecta también a los usuarios que se creen después.
6. `GRANT SELECT ON ALL TABLES IN SCHEMA ventas TO lectura;` incluye las tablas que se creen más adelante en ese esquema.
7. En PostgreSQL 15 o posterior, un usuario recién creado puede crear tablas en el esquema `public`.
8. Con RLS activo y sin políticas, el dueño de la tabla tampoco puede ver sus filas.
9. `REVOKE ventas_lectura FROM ana;` elimina los privilegios que tenía el rol `ventas_lectura`.
10. Virtual Private Database (Oracle) y Row Level Security (PostgreSQL) resuelven el mismo problema; la diferencia principal está en cómo se expresa el predicado.